

**MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA**

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory Work No. 1 and No. 2

Operating Systems

Student: Bianca-Andreea Tataroi
Group: FAF-242

Checked by: Reițman Patricia

Chișinău – 2026

Cuprins

1 Laboratory Work No. 1 **2**
1.1 The OS as a Resource Manager . . . 2
1.2 Objective . . . 2
1.3 Initial System Information . . . 2
1.4 Part 1 – Files and Directories . . . 2
1.4.1 Observations . . . 3
1.5 Part 2 – Process Management . . . 3
1.5.1 Observations . . . 3
1.6 Part 3 – Memory Management . . . 3
1.7 Part 4 – Devices and Storage . . . 4
1.8 Conclusion for Laboratory Work No. 1 . . . 4

2 Laboratory Work No. 2 **5**
2.1 Meet the OS You Will Build – xv6 . . . 5
2.2 Objective . . . 5
2.3 Part 1 – Building and Booting xv6 . . . 5
2.4 Part 2 – Exploring xv6 . . . 5
2.5 Part 3 – Source Code Analysis . . . 6
2.6 Part 4 – Creating the sleep Program . . . 6
2.7 Conclusion for Laboratory Work No. 2 . . . 7

3 General Conclusion **7**

1 Laboratory Work No. 1

1.1 The OS as a Resource Manager

1.2 Objective

The objective of this laboratory work was to explore how the Linux operating system manages its main resources: files, processes, memory and devices.

The laboratory was performed using a Linux virtual machine and basic terminal commands.

1.3 Initial System Information

The following commands were used to obtain information about the current system:

```
whoami
uname -a
uptime
```

The current user was:

```
bianca
```

The system was running a Linux kernel and the uptime command showed that the virtual machine had been running for several minutes.

1.4 Part 1 – Files and Directories

The following commands were used to explore the file system:

```
pwd
ls -la /
ls -la ~
cd /etc
ls | head
```

A new directory and several files were then created:

```
cd ~/os-lab1
mkdir demo
cd demo

echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
```

File permissions were changed using:

```
chmod 600 note.txt
ls -l note.txt
chmod 644 note.txt
```

After using `chmod 600`, the result was similar to:

```
-rw------- 1 bianca bianca 24 Sep 28 note.txt
```

1.4.1 Observations

1. The files were owned by the user `bianca` and belonged to the group `bianca`.
2. The permissions `-rw----` mean that the file is a regular file and only the owner can read and write it.
3. The directory `/home` contains users' personal directories, while `/etc` contains system configuration files.

1.5 Part 2 – Process Management

The running processes were inspected using:

```
ps aux | head
ps aux | wc -l
top
```

A background process was created:

```
sleep 300 &
jobs
ps -ef | grep sleep
```

The process was then stopped with the `kill` command.

The Linux `/proc` directory was also used to inspect information about the process:

```
sleep 300 &
PID=$!
cat /proc/$PID/status | head -20
kill $PID
```

The state of the process was:

```
State: S (sleeping)
```

1.5.1 Observations

PID 1 is the first process started by the system and is responsible for starting and managing system services.

The `sleep` process had the state `S (sleeping)`, meaning that it was waiting and was not actively using the CPU.

1.6 Part 3 – Memory Management

Memory information was inspected using:

```
free -h
cat /proc/meminfo | head -6
```

The virtual machine had approximately:

- Total RAM: 3.3 GiB
- Free RAM: approximately 160 MiB
- Swap: 0 B

The resident memory used by a simple `sleep` process was checked with:

```
sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
```

The process used approximately:

```
VmRSS: 6356 kB
```

Swap is disk space that can be used as additional virtual memory when physical RAM becomes insufficient.

1.7 Part 4 – Devices and Storage

The following commands were used:

```
df -h
lsblk
du -sh ~/os-lab1
ls -l /dev | head
mount | head
```

The root filesystem was mounted on:

```
/dev/sda2
```

The laboratory directory occupied approximately:

```
16K
```

One example of a device entry is `/dev/sr0`, which represents an optical CD/DVD device.

The UNIX principle "everything is a file" means that hardware devices and other system resources can be accessed through file-like interfaces.

1.8 Conclusion for Laboratory Work No. 1

The operating system manages four important types of resources: files, processes, memory and devices.

The command `ls` was used to inspect files, `ps` to inspect processes, `free` to inspect memory and `lsblk` to inspect storage devices.

This laboratory demonstrated how Linux provides a unified interface for managing the computer's resources.

2 Laboratory Work No. 2

2.1 Meet the OS You Will Build – xv6

2.2 Objective

The objective of this laboratory work was to become familiar with the xv6 educational operating system running on the RISC-V architecture.

The laboratory included compiling and running xv6 in QEMU, using basic Unix commands, examining the source code and creating a new user program.

The xv6 operating system is a small Unix-like teaching operating system that makes it possible to study both user-space programs and kernel code. :chatgpt-content-referenceindex="0"

2.3 Part 1 – Building and Booting xv6

The xv6 source directory was opened using:

```
cd ~/xv6-riscv
ls
```

The main directories include:

- `kernel/`
- `user/`
- `Makefile`

The operating system was compiled and started with:

```
make qemu
```

After booting, xv6 displayed:

```
xv6 kernel is booting

init: starting sh
$
```

The laboratory specifically uses QEMU to boot the compiled xv6 kernel. :chatgpt-content-referenceindex="1"

2.4 Part 2 – Exploring xv6

Inside the xv6 shell, the following commands were tested:

```
ls
cat README
echo hello xv6
ls | grep c
wc README
```

Examples of programs included in xv6 are:

- `ls`
- `cat`
- `echo`
- `grep`
- `wc`

A pipe requires multiple processes and a mechanism for communication between those processes.

The xv6 shell is similar to a Linux shell, but it is much simpler and contains only the basic functionality required for teaching operating system concepts. :chatgpt-content-referenceindex="2"

2.5 Part 3 – Source Code Analysis

The source code of several user programs was inspected using:

```
sed -n "1,40p" user/ls.c
sed -n "1,30p" user/cat.c
cat user/user.h
grep -n "sys_read\|sys_write" kernel/sysfile.c | head
```

The program `cat.c` uses system calls such as:

- `read()` – requests data from a file or input source;
- `write()` – sends data to a file or output source;
- `exit()` – terminates the current process.

The implementation of `sys_read` can be found in:

```
kernel/sysfile.c
```

The exact line number can be found using the `grep` command above.

Code inside `kernel/` executes with privileged access and implements operating system services. Code inside `user/` contains ordinary programs that request these services through system calls. :chatgpt-content-referenceindex="3"

2.6 Part 4 – Creating the sleep Program

A new user program called `sleep.c` was created inside the `user/` directory.

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    if(argc != 2){
        fprintf(2, "usage:␣sleep␣<ticks>\n");
```

```
        exit(1);
    }

    sleep(atoi(argv[1]));
    exit(0);
}
```

The program checks whether the correct number of arguments was supplied. It converts the argument to an integer and passes it to the `sleep()` system call.

The following entry was added to the `UPROGS` section of the Makefile:

```
$U/_sleep\
```

The program was then tested using:

```
make qemu
```

Inside xv6:

```
sleep 10
sleep
```

Running `sleep 10` pauses the process for ten clock ticks.

Running `sleep` without an argument displays:

```
usage: sleep <ticks>
```

This task creates a user program that invokes the existing xv6 `sleep` system call. :chatgpt-content-referenceindex="4"

2.7 Conclusion for Laboratory Work No. 2

During this laboratory work, the xv6 operating system was compiled and executed in the QEMU emulator.

Basic Unix commands were tested, and the difference between user-space programs and kernel code was examined.

Finally, the `sleep` utility demonstrated how a user program can request a service from the xv6 kernel through a system call.

3 General Conclusion

The two laboratory works provided a practical introduction to the main concepts of operating systems, both from the user's perspective and from the internal structure of the operating system itself.

In the first laboratory work, the Linux operating system was explored as a resource manager. The main resources managed by the operating system were files, processes, memory and hardware devices. Commands such as `ls`, `ps`, `free`, `lsblk` and other terminal utilities were used to observe how these resources are organized and controlled. File ownership and permissions were also analyzed, showing how Linux controls access to files and directories. The work with processes demonstrated how programs are created, monitored and terminated, while the memory section showed how physical memory and process memory usage can be inspected. The storage and device section demonstrated the UNIX principle that many system resources are represented through file-like interfaces.

In the second laboratory work, the focus moved from using an operating system to understanding how one is organized internally. The xv6 educational operating system was compiled and executed using the QEMU emulator. Basic commands such as `ls`, `cat`, `echo`, `grep` and `wc` were tested inside the xv6 shell, showing that xv6 follows many of the same Unix concepts as Linux.

The source code analysis helped to understand the separation between `kernel/` and `user/`. Programs located in the user space run with limited privileges and request services from the kernel through system calls such as `read()`, `write()` and `sleep()`. In contrast, kernel code runs with privileged access and is responsible for managing system resources and responding to these requests.

The implementation of the `sleep` utility was especially useful because it demonstrated the complete interaction between a user program and the operating system kernel. The program receives an argument from the user, converts it into clock ticks and calls the existing `sleep()` system call provided by xv6. This made the concept of a system call more concrete and showed how user applications depend on kernel services.

Overall, these laboratory works created a clear connection between the external behavior of an operating system and its internal implementation. Linux was used to observe how a real operating system manages resources, while xv6 made it possible to study the source code responsible for similar operations. As a result, the laboratories provided a better understanding of the role of the operating system, the distinction between user space and kernel space, and the communication between applications and the kernel.